

Topic 1: Recursive Problem Solving

Section	Details
Definition	Recursion is a programming technique where a function calls itself to solve smaller instances of the same problem until a base condition is reached.
Importance	Simplifies complex problems, models divide-and-conquer algorithms, and forms the basis of many advanced algorithms.
Real-Life Examples	File/folder traversal, family trees, organizational hierarchies, nested comments, mathematical computations.
Key Concepts	Base Case, Recursive Case, Function Call Stack
Complexity	Depends on the recursive relation (e.g., $O(n)$, $O(2^n)$).
Best Practices	Always define a base case, reduce problem size each call, avoid infinite recursion.

Classroom Demo 1

Factorial using Recursion

```
int factorial(int n)
{
    if(n==0)
        return 1;

    return n * factorial(n-1);
}
```

Demo Flow

factorial(5)

↓

$5 \times \text{factorial}(4)$

↓

$4 \times \text{factorial}(3)$

↓

...

↓

factorial(0)=1

Topic 2: Tail Recursion

Section	Details
Definition	A recursive function where the recursive call is the last operation performed before returning.
Importance	Easier for compiler optimization (tail-call optimization, where supported), reduces stack overhead conceptually.
Real-Life Example	Countdown timers, linear traversals, accumulating sums.
Difference from Normal Recursion	No computation remains after the recursive call returns.
Advantages	Cleaner logic for linear recursive problems and potential optimization in some languages/compilers.

Demo

```
int factorialTail(int n,int result)
{
    if(n==0)
        return result;

    return factorialTail(n-1,n*result);
}
```

Activity

Lets compare:

Recursive Factorial	Tail Recursive Factorial	Iterative Factorial
<pre>int factorial(int n) { if(n==0) return 1; return n * factorial(n-1); }</pre>	<pre>int factorialTail(int n,int result) { if(n==0) return result; return factorialTail(n-1,n*result); }</pre>	Using for loop

Discuss

- Number of function calls
- Readability
- Memory usage
- Performance considerations

Topic 3: Backtracking Concepts

Section	Details
Definition	Backtracking is an algorithmic technique that incrementally builds a solution and abandons a path as soon as it violates constraints.
Importance	Solves combinatorial and constraint satisfaction problems efficiently.
Real-Life Examples	Puzzle solving, route planning, password generation, scheduling, game AI.
Core Steps	Choose → Explore → Validate → Backtrack
Data Structures	Arrays, Recursion, Stack (Call Stack)

Backtracking Workflow

Choose
↓
Explore
↓
Valid?
↓
Yes → Continue
↓
No
↓
Undo Choice
↓
Try Next Option

Demo Activity

Generate all binary strings of length 3.

Expected Output

000
001
010
011
100
101
110
111

Students observe how recursion explores and backtracks.

Current Position	Try	Result	Next Step
Position 1	0	Continue	Go deeper

Position 2	0	Continue	Go deeper
Position 3	0	String Complete	Print 000
Position 3	1	String Complete	Print 001
Position 2	Done	Backtrack	Change previous choice
Position 2	1	Continue	Explore new branch
Position 3	0	String Complete	Print 010
Position 3	1	String Complete	Print 011
Position 1	Done	Backtrack	Choose 1 and repeat

Function Call	Stack State
generate("",0)	generate(0)
generate("0",1)	generate(0) → generate(1)

generate("00",2)	generate(0) → generate(1) → generate(2)
generate("000",3)	generate(0) → generate(1) → generate(2) → generate(3)
Print 000	Pop one call
generate("001",3)	Push again
Print 001	Pop one call
Return to 01	Continue exploring

Topic 4: N-Queens Problem

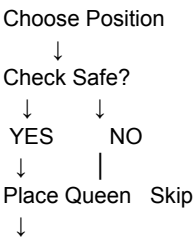
Section	Details
Definition	Place N queens on an N×N chessboard so that no two queens attack each other.
Importance	Classic constraint satisfaction and backtracking problem.
Real-Life Applications	Scheduling, resource allocation, optimization problems.
Constraints	No two queens in the same row, column, or diagonal.

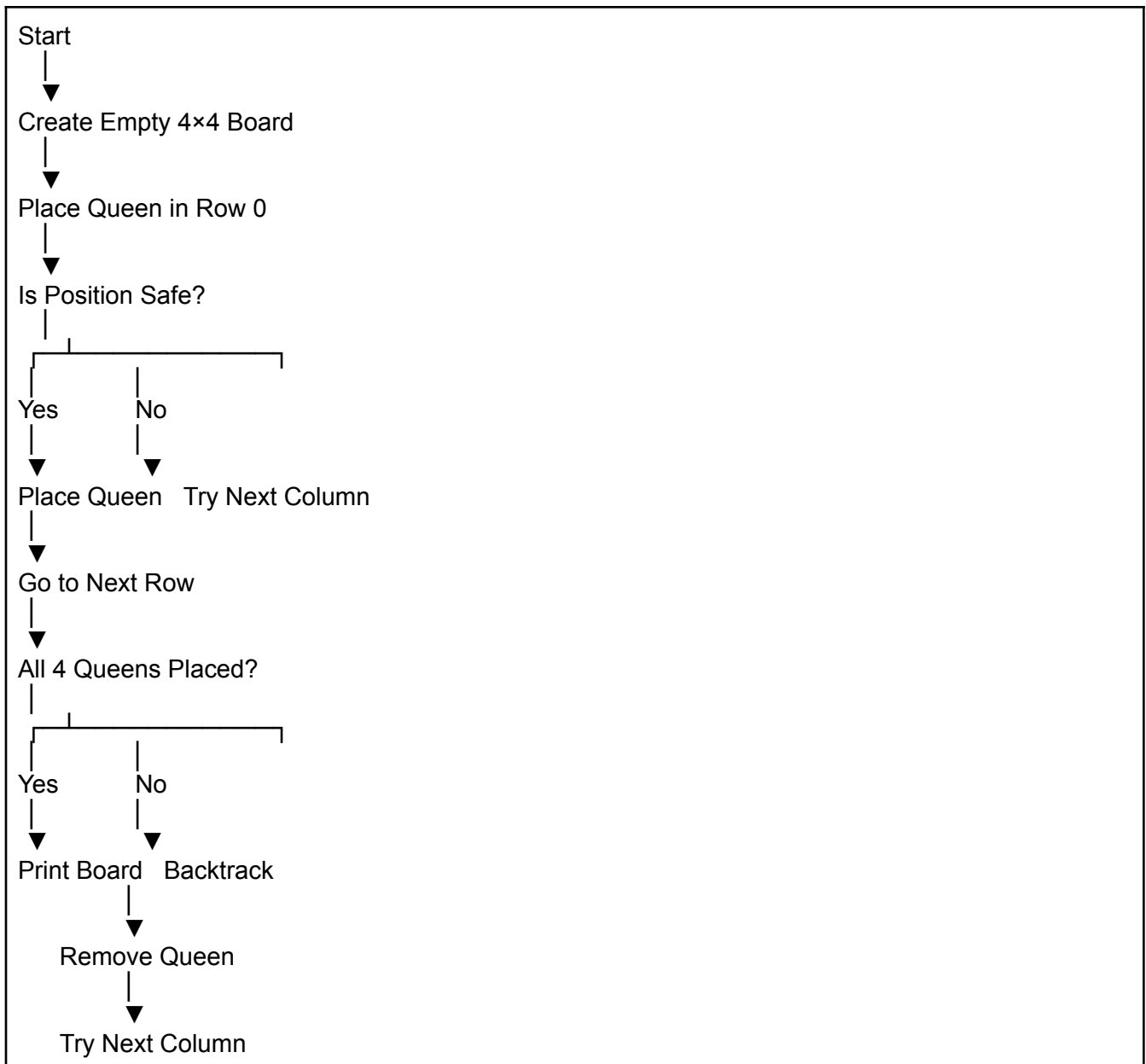
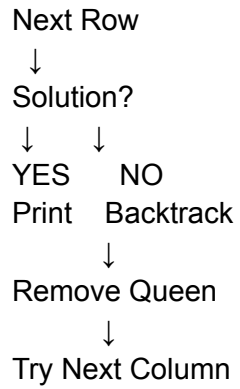
Demo (4 × 4)

	Queen		
			Queen
Queen			
		Queen	

Q . . .
 . . Q .
 . Q . .
 . . . Q
 •

Step	Question to Ask	Student Response
Choose	Which column should I try?	Pick a column
Check	Is it safe?	Yes / No
Explore	Can I place a queen in the next row?	Recurse
Backtrack	No valid move left?	Remove queen and try the next column





<https://queens-gambit-visualizer.vercel.app>
<https://visualgo.net/en>

Coding Concepts

- 2D Arrays
 - Functions
 - Recursion
 - Backtracking
-

Topic 5: Maze Solving

Section	Details
Definition	Find a valid path from the start position to the destination in a maze using recursive exploration and backtracking.
Importance	Introduces graph traversal concepts through recursion.
Applications	GPS navigation, robotics, AI path finding, games.

Demo Maze

S 1 0 0

1 1 1 0

0 1 0 1

0 1 1 E

Explain

Move Right



Move Down



Blocked?



Backtrack



Try Another Direction

Concepts Covered

- Recursive DFS
 - Boundary Checking
 - Visited Cells
 - Path Recording
-

Topic 6: Sudoku Solver Basics

Section	Details
Definition	Fill empty cells while satisfying Sudoku rules using recursive backtracking.
Importance	Demonstrates recursive search with multiple constraints.
Applications	Puzzle solving, AI search techniques, constraint programming.

Demo (4 × 4 Sudoku)

1 _ _ 4

_ 3 _ _

_ _ 2 _

2 _ _ 3

Explain

Find Empty Cell

↓

Try Number

↓

Valid?

↓

Yes

↓

Recurse

↓

Wrong?

↓

Undo

↓

Try Next Number

Concepts Covered

- Constraint Checking
 - Recursive Search
 - Backtracking
 - Validation Functions
-

Practice Activities

Activity	Duration	Objective
Trace Factorial Function Calls	10 min	Understand recursion stack
Dry Run Fibonacci	10 min	Identify repeated subproblems
Predict Recursive Output	10 min	Improve recursion logic
Solve Small Maze on Paper	15 min	Understand backtracking
Place 4 Queens	15 min	Apply constraint checking
Solve 4×4 Sudoku	20 min	Practice recursive decision making

Demo Practice

Demo	Duration
Recursive Factorial	10 min
Fibonacci Recursion	10 min
Tail Recursion	10 min
Binary String Generator	10 min
N-Queens (4×4)	25 min
Maze Solver	20 min
Sudoku Solver (4×4)	20 min

Assessment Questions

Difficulty	Questions
Easy	Define recursion.

Easy	What is a base case?
Easy	What is tail recursion?
Medium	Compare recursion and iteration.
Medium	Explain backtracking with an example.
Medium	Why is backtracking suitable for N-Queens?
Difficult	Trace the recursion tree for Fibonacci(5).
Difficult	Explain the time complexity of recursive Fibonacci.
Difficult	Describe how a maze solver uses backtracking.
Difficult	Explain the algorithm of a Sudoku solver.

Learning Outcomes Mapping

Outcome	Topics Covered
Explain recursion and recursive thinking	Recursive Problem Solving
Implement recursive algorithms	Factorial, Fibonacci
Differentiate recursion types	Tail Recursion
Apply backtracking	Binary Strings, N-Queens
Solve constraint satisfaction problems	N-Queens, Sudoku
Solve path-finding problems	Maze Solver
Analyze recursive execution and complexity	All topics

Mini Assignment (Post-Session)

Students should implement any **three** of the following:

1. Recursive Power Function (x^n)
2. Recursive Sum of Digits

3. Recursive Binary Search
4. Tower of Hanoi
5. Rat in a Maze (4×4)
6. N-Queens (4×4)
7. 4×4 Sudoku Solver (basic)
8. Generate All Subsets of a String

Step	Action	Questions to Ask Yourself
1	Read the Problem	What exactly is being asked?
2	Understand Input & Output	What are the constraints?
3	Solve Manually	Can I solve it using paper and pencil?
4	Identify the Pattern	Array, Stack, Tree, Graph, Recursion?
5	Choose the Data Structure	Which data structure fits best?
6	Write the Algorithm	Explain the steps before coding.
7	Dry Run	Test with a small example.
8	Write the Code	Convert the algorithm into code.
9	Test & Debug	Try edge cases and boundary values.
10	Optimize	Can I improve the time or space complexity?





Submission	10 Demo based on First 5 days of topics First week C Programming & Data Structures Lab Assignments.pdf
Mini Project presentation	Presentation/VIVA for 10-15 mins
	WORKING CODE IN .c PPt PRESENTATION FOR ABOVE MINI PROJECT Code uploaded on github With screenshots of at least 5 user stories Word report as per template

TOPIC SUMMARY

1 RECURSIVE PROBLEM SOLVING

Definition:

A function calls itself to solve smaller instances of the same problem until a base condition is reached.

Importance:

Simplifies complex problems, models divide-and-conquer algorithms.

Use Cases:

- Mathematical computations
- File/Folder traversal
- Tree/Graph problems
- Nested structures

Demo: Factorial

```
int factorial(int n)
{
    if(n==0)
        return 1;
    return n * factorial(n-1);
}
```

Flow (factorial(5))

5 → 4 → 3 → 2 → 1 → 0
(return 1 and multiply back)

2 TAIL RECURSION

Definition:

The recursive call is the last operation before returning.

Importance:

Can be optimized by compiler (Tail Call Optimization).

Use Cases:

- Linear computations
- Count-down problems
- Accumulator based problems

Demo: Tail Recursive Factorial

```
int factTail(int n, int res)
{
    if(n==0)
        return res;
    return factTail(n-1, n*res);
}
```

More efficient in memory
(for supported compilers)

3 BACKTRACKING CONCEPTS

Definition:

An algorithmic technique that builds a solution incrementally and abandons a path when it violates constraints.

Importance:

Solves combinatorial and constraint satisfaction problems efficiently.

Use Cases:

- Puzzle solving
- Game playing (AI)
- Route planning
- Scheduling, Passwords

Core Steps:

- Choose
- Explore
- Validate
- If Invalid → Backtrack
- Try Next Option

4 N-QUEENS PROBLEM

Definition:

Place N queens on an N×N chessboard such that no two queens attack each other.

Importance:

Classic constraint satisfaction problem.

Constraints:

- No two queens in same row
- No two queens in same column
- No two queens in same diagonal

Demo (4×4 Solution):

```
Q . . .
. . Q .
. Q . .
. . . Q
```

Concepts:

2D Arrays, Recursion, Backtracking, Safety Check

5 MAZE SOLVING

Definition:

Find a path from start (S) to end (E) in a maze using recursive exploration and backtracking.

Importance:

Introduces graph traversal and decision making.

Use Cases:

Robotics, GPS, Games, Path finding

Demo Maze:

```
S 1 0 0
1 1 1 0
0 1 0 1
0 1 1 E
```

1 → Open Path
0 → Blocked Path
S → Start E → End

Concepts:

DFS, Backtracking, Boundary Check, Visited Cells

6 SUDOKU SOLVER BASICS

Definition:

Fill empty cells while satisfying Sudoku rules using recursive backtracking.

Importance:

Demonstrates recursive search with multiple constraints.

Demo (4×4 Sudoku):

```
1 . . 4
. 3 . .
. . 2 .
2 . . 3
```

Concepts:

Constraint Checking, Recursion, Backtracking, Validation Functions

WHY RECURSION & BACKTRACKING ARE IMPORTANT?



Solves complex problems easily



Forms basis of many advanced algorithms



Used in AI, Game Development, Robotics



Improves logical thinking and problem solving

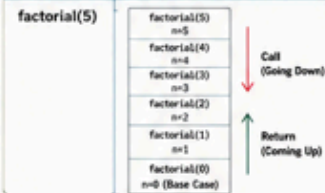


Essential for competitive programming

RECURSION vs ITERATION

Feature	Recursion	Iteration
Approach	Function calls itself	Uses loops
Memory	Uses Call Stack	Constant/Low
Readability	More intuitive for problems like Trees, DFS, Backtracking	Better for linear simple tasks
Risk	Stack Overflow (if deep recursion)	No Stack Overflow
Performance	May be slower	Usually faster

RECURSION CALL STACK (Example)



LIVE CODING DEMOS

- Recursive Factorial
- Fibonacci Series
- Tail Recursion
- Generate Binary Strings
- N-Queens (4×4)
- Maze Solver
- Sudoku Solver (4×4)

CLASSROOM ACTIVITIES

- Trace Factorial Function Calls
- Dry Run Fibonacci
- Predict Recursive Output
- Solve Small Maze on Paper
- Place 4 Queens
- Solve 4×4 Sudoku

ASSESSMENT QUESTIONS

- Easy**
- Define recursion.
 - What is a base case?
- Easy**
- What is tail recursion?
- Medium**
- Compare recursion and iteration.
 - Explain backtracking with an example.
- Medium**
- Why is backtracking suitable for N-Queens?
- Difficult**
- Trace the recursion tree for Fibonacci(5).
 - Explain the time complexity of recursive Fibonacci.
 - Describe how a maze solver uses backtracking.
 - Explain the algorithm of a Sudoku solver.

MINI ASSIGNMENT (Do any 3)

- Recursive Power Function (xⁿ)
- Recursive Sum of Digits
- Recursive Binary Search
- Tower of Hanoi
- Rat in a Maze (4×4)
- N-Queens (4×4)
- 4×4 Sudoku Solver (basic)
- Generate All Subsets of a String

BEST PRACTICES FOR RECURSION

- Always define a base case.
- Ensure the problem size reduces in each call.
- Avoid unnecessary global variables.
- Analyze time and space complexity.
- Use meaningful function names.

COMMON MISTAKES

- Missing or incorrect base case
- Infinite recursion
- Not reducing problem size
- Excessive recursion (Stack Overflow risk)
- Unnecessary recomputation

KEY TAKEAWAY

Recursion helps you break complex problems into smaller sub-problems, while backtracking helps you explore all possible solutions efficiently by undoing wrong choices. Mastering these techniques is essential for advanced problem solving and real-world applications.



"Every problem has a recursive solution. You just need the right base to start."

5 MAZE SOLVING

Definition:

Find a path from start (S) to end (E) in a maze using recursive exploration and backtracking.

Importance:

Introduces graph traversal and decision making.

Use Cases:

Robotics, GPS, Games,
Path finding

Demo Maze:

S	1	0	0
1	1	1	0
0	1	0	1
0	1	1	E

1 → Open Path

0 → Blocked Path

S → Start E → End